

Step 1: Getting the GeoJSON data

Our map will be created from data downloaded from the public domain portal *Natural Earth*. This site provides GIS data in vector and raster formats with shapefiles for cultural and physical features that can be used in any project for free. You can also apply these steps to shapefile data you may download from other sources.

Natural Earth provides several datasets. For this example, we will use features that represent countries by their political, or cultural, borders. Since we will represent the entire planet, we can use a low-resolution dataset, which is smaller. Follow the steps shown in *Figure 1* to download it.

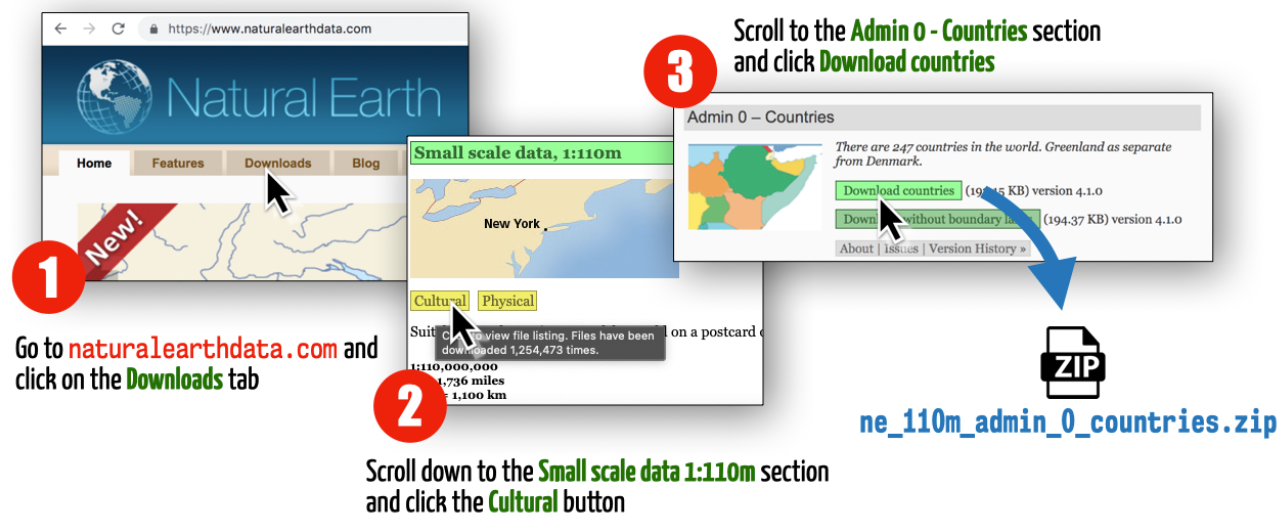


Figure 1 – Downloading a shapefile for a world map from [naturalearthdata.com](https://www.naturalearthdata.com).

This will download a ZIP file (see Step01/ne_110m_admin_0_countries.zip) containing shapefiles in the standard ESRI Shapefile format that can be edited using a GIS application, such as QGIS.

To use this file in D3, we need it in the *GeoJSON 2008* format. There are many ways to do this:

- Load the file in a free QGIS application and export it as GeoJSON 2008.
- Use the ogr2ogr command line tool from gdal.org to create a GeoJSON file from the SHP file. It generates GeoJSON 2008 as default.
- Use a free online service such as mapshaper.org or geojson.io.

The last option is probably the simplest. Let's try mapshaper.org: open the main page and drop your ZIP file in the main window. It should instantly render all the shapes in your file, showing a cylindrical projection of the world (see *Figure 2*). Then click on the *Export*

tab (upper-left) to open the *Export options* dialog. Select *GeoJSON* with and type *gj2008* in the command-line options box, to export a GeoJSON version of the shapefile compatible with the *d3-geo* library.

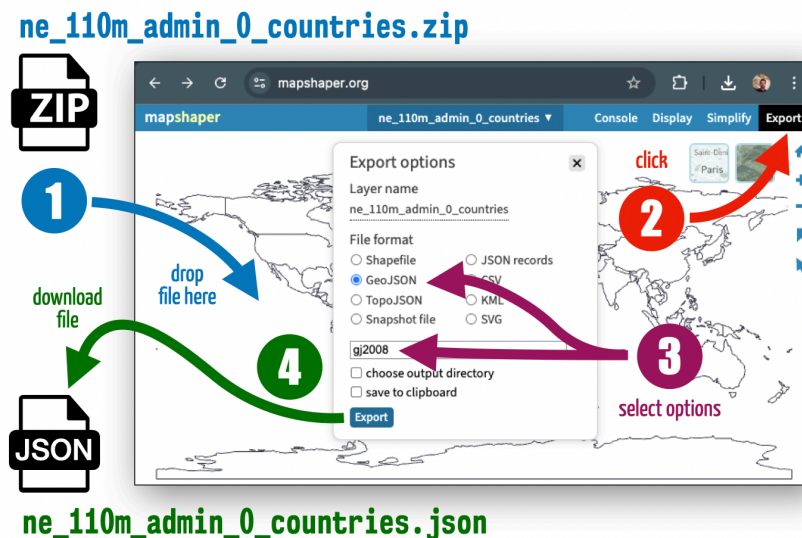


Figure 2 – Converting data in Shapefile format (SHP) to GeoJSON using mapshaper.org.

You can also use *geojson.io* to convert Shapefiles or other GIS formats to GeoJSON and vice-versa. It allows you to inspect and edit the map and code. After dropping the file in the app's main window, you will see the rendered file in the center and the JSON code at right. Click on the *Export* tab to download different GIS formats.

It's not always so simple. You may encounter files that are using a different coordinate system (the standard, supported by D3, is *WGS 84*), winding problems, errors and other inconsistencies. In these cases, you may need to fix, convert or prepare the data using a GIS app or script.

Important

You must add the *gj2008* command line option when exporting GeoJSON from mapshaper.org, as D3 follows the *GeoJSON 2008* specification, *not* the most recent *RFC7946* spec (2016), which is adopted by mapshaper.org. The new spec generates coordinates in a different polygon winding order, which may render shapes incorrectly when used with *d3-geo* functions. Be aware that this may change in future versions of D3.

If you are exporting via *geojson.io*, which has also recently updated to *RFC7946*, click *Advanced options* in the *Export 531 features* dialog, and select *D3* as the winding order to use the 2008 spec.

Step 2: Preparing a GeoJSON dataset

Open the GeoJSON file you generated in the previous step (or use the copy in Step02/data/ne_110m_admin_0_countries.json) to inspect its structure. It contains a single FeatureCollection object with a single array property called features. In this file each feature is a country or autonomous region.

Here is a snippet highlighting the first entry in the features array from the GeoJSON file. It represents the Fiji archipelago, which in this map file consists of three polygons.

```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": {
        "type": "MultiPolygon",
        "coordinates": [
          [[[180, -16.1], ... , [180, -16.1]]],
          [[[178.1, -17.5], ... , [178.1, -17.5]]],
          [[[-179.8, -16], ... , [-179.8, -16]]] ]
        },
      "properties": {
        "NAME": "Fiji",
        "POP_EST": 889953,
        "POP_YEAR": 2019,
        "GDP_MD": 5496,
        "GDP_YEAR": 2019,
        "ADMO_A3": "FJI",
        "CONTINENT": "Oceania",
        "SUBREGION": "Melanesia",
        /* ... 159+ properties not shown ... */
      }
    }, /* ... 175+ Feature objects not shown ... */
    {
      "type": "Feature",
      "geometry": {...},
      "properties": {...}
    }
  ]
}
```

The geometry object can be a Polygon or a MultiPolygon type. The coordinates array contain *[longitude, latitude]* pairs that represent the border shape of a country.

Continuous territories are of type Polygon. If a country has islands or separate territories, it will be a MultiPolygon, which is an array of polygons, and its coordinates array will be one level deeper. You don't need to worry about this distinction when using *d3-geo*.

The properties key, in each feature, contains over 160 properties. These include the country's name and code, but also lots of information you won't need, like names and codes in multiple locales.

Normally, when you download shapefiles, you need to prepare them for the Web. GIS apps like QGIS offer a lot of interactive tools that you can use to convert coordinate systems, fix

overlapping lines, optimize and simplify your shapes, and manage the properties you include for each feature. In this case, we will use a script to remove the properties we don't need.

The following Node.js script (see Tools/FeatureFilter.js) creates a smaller data file (world-lowres.geojson) preserving name, continent, population, and the country's ISO 3166 alpha-3 code stored as the standard id property:

```
import * as fs from "fs";

const DIR = "../StepByStep/Step02/data";
const inputFile = DIR + "/ne_110m_admin_0_countries.json";
const outputFile = DIR + "/world-lowres.geojson";
const data = JSON.parse(fs.readFileSync(inputFile, "utf-8"));

data.features = data.features.map(feature => {
  feature.id = feature.properties.ADM0_A3;
  feature.properties = {
    continent: feature.properties.CONTINENT,
    name: feature.properties.NAME,
    pop: feature.properties.POP_EST
  };
  return feature;
});
fs.writeFileSync(outputFile, JSON.stringify(data), "utf-8");
```

If you run the script from the Tools/ folder, it will store the result in Step02/data/. The following snippet shows the result, highlighting the first entry, after running the script:

```
{"type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "id": "FJI",
      "geometry": {
        "type": "MultiPolygon",
        "coordinates": [ ... ]
      },
      "properties": {
        "name": "Fiji",
        "pop": 889953,
        "continent": "Oceania"
      }
    },
    /* ... 175+ feature objects not shown ... */
    {
      "type": "Feature",
      "id": "SDS",
      "geometry": { ... },
      "properties": { ... }
    }
  ]
}
```

Let's test the world-lowres.geojson file creating a page that loads the file and prints a list of country names, codes and population. Create a simple HTML page calling the module that will contain all the code, for example js/main.js (see the full page in Step02/index.html):

```
<script type="module" src="js/main.js"></script>
```

Now set up the main.js module, containing the code. This will load the D3 library, load the GeoJSON data file and use a data join to bind each feature to an element, which

prints the feature's id property and name and pop, from the properties property (see Step02/js/main.js):

```
import * as d3 from "https://cdn.jsdelivr.net/npm/d3@7/+esm";
const data = await d3.json("data/world_lowres.geojson");
d3.select("body").append("ul")
  .selectAll("li")
  .data(data.features)
  .join("li").text(d => `${d.properties.name} (${d.id):
                        population: ${d.properties.pop}`);
```

Open index.html to see an HTML file listing the names, code and population of all the countries represented by this GeoJSON file.

Step 3: Drawing a map

The *d3-geo* library makes it much easier to create sophisticated maps. You can quickly render data from a GeoJSON file with just a few lines of code and choose among over one hundred different map projections. In this section we will ...

The functions and methods we will use are listed in *Table 1*.

Function or method	Description
<code>geoPath = d3.geoPath()</code>	Creates a layout function which generates an SVG path string from a GeoJSON object.
<code>geoPath.projection(projection)</code>	A projection function converts geographical coordinates into pixels projecting spherical coordinates to a flat surface. You can choose among over a hundred different projections. The default treats latitudes and longitudes as pixel coordinates.
<code>geoPath.context(canvasCtx)</code>	If a canvas context is set, instead of returning an SVG path string the layout function will return a list of canvas path operations.
<code>geoPath.pointRadius(radius)</code>	Point and MultiPoint GeoJSON objects are rendered as small circles (using path commands). This method configures their radii.

Table 1 – Geographical path generator and its main configuration methods.

The main shape generator is the `d3.geoPath()` function. It returns a function that takes a GeoJSON object and creates a path within the bounds dictated by a map projection function. The generated string can be passed to the `d` attribute of a `<path>` element.

Like other SVG path string generators, `d3.geoPath()` can also be used to draw in HTML Canvas. If it is initialized with a Canvas context via the `context()` method.

Let's render a map with the default projection. First, create an SVG viewport. The dimensions of 960x500 are perfect to render a cylindrical projection of the earth:

```
const dim = {width: 960, height: 500}
const svg = d3.select("body").append("svg")
    .attr("width", dim.width).attr("height", dim.height);
```

Simply call `d3.geoPath()` to generate a `geoPath` function with default parameters:

```
const geoPath = d3.geoPath();
```

Call it in a data join to create the SVG path string:

```
svg.selectAll("path.country")
    .data(data.features)
    .join("path").attr("class", "country")
    .attr("d", geoPath);
```

The `d` attribute for each `<path>` is generated calling `geoPath(feature)` for each feature object. It locates the `geometry` array in the feature and uses longitudes and latitudes to create the path string in screen coordinates.

Screen coordinates mean that you will only see the quarter of the map with positive longitudes and latitudes, as the origin, where the prime meridian and equator cross, will be placed at the top-left corner of the viewport. The drawing will also be 180 pixels wide and 90 pixels high, since the default identity projection treats one pixel as one degree.

We can, of course, fix this using scales, the SVG `viewBox` or affine transforms, as we have done in *Chapters 2* and *4*, but it's much easier using projections.

Step 4: Choosing a projection

The *d3-geo* library comes with a collection of popular projection functions, which can be used to convert any longitude-latitude pair to screen coordinates. For example, this will create a Mercator function using default settings:

```
const projection = d3.geoMercator();
```

If you have a latitude-longitude pair, you can plot it in the viewport at a relative position according to the current projection using the x-y pair returned by calling the function. For example:

```
const mexico_city = [-99.1332, 19.4326]; // longitude, latitude  
const position = projection(mexico_city); // returns [473.644, 229.769]
```

The following code uses these points to draw a dot showing where Mexico City appears in the map. Add this at the end of the file, so it appears above any shapes:

```
svg.append("circle")  
  .attr("cx", position[0]).attr("cy", position[1]).attr("r", 5)  
  .style("fill", "red");
```

Using the projection function when setting up *geoPath* will render all shapes according to the Mercator projection:

```
const geoPath = d3.geoPath().projection(projection);
```

Reload the page and you should now see the world map using all the space in your viewport. The red dot marks the position of Mexico City.

GeoJSON files may also contain Point and LineString types, which are also generated automatically with *d3.geoPath()*. If your file has Point features, you must define the radius of the circle used to represent them when you set up a *geoPath*:

```
const geoPath = d3.geoPath().pointRadius(3); // Points are rendered with 3-pixel radii
```

The *d3-geo* module comes with 11 other projections you can use the way Mercator was used here (there are many more in the *d3-geo-projection* module). Try replacing *d3.Mercator()* with *d3.geoOrthographic()* to see a globe-like projection of the same data.

Step 5: Making a Choropleth

A choropleth is a thematic map where geographic areas are shaded or colored in proportion to a statistical variable. To create a choropleth, we need to combine shape data, obtained from the GeoJSON file, with thematic data, usually obtained from other sources unless it's already in your features' properties object.

Our data file contains population data (from 2016) for all features, in the `properties.pop` property. Let's use it to create a choropleth that compares populations.

To set up a color scale we will use red hues for high populations and blue hues for low populations, with yellow in the middle. This spectrum is available in the `d3.interpolateRdYlBu` color scheme (from the *d3-scheme-chromatic* module – see *Chapter 10*).

To set up a scale, we need to compute the extent of the data, used to set the domain.:

```
const extent = d3.extent(data.features, d => d.properties.pop);
```

Since population differences span many orders of magnitude, using a linear scale will only highlight the two most populous countries, and differences between the others will be hard to distinguish. An exponential scale with a fractional exponent can be used to pull the median closer to the lower values, allowing you to spot more differences between countries.

```
const color = d3.scaleSequentialPow(d3.interpolateRdYlBu).exponent(.2)
                  .domain(extent.reverse());
```

The domain was reversed because the color scheme starts in red and ends in blue, and we want the opposite effect.

Now call this `color()` function in a selection containing all the features, using the population to obtain a color to fill the shape:

```
svg.selectAll("path.country") // CSS styles defined above
    .data(data.features)
    .join("path").attr("class", "country")
    .attr("d", geoPath)
    .style("fill", d => color(d.properties.pop));
```

Making a choropleth with the country's population was easy, because the data was already in the file, but what if your data is somewhere else? We will deal with this problem in the next step.

Step 6: Adding more themes

You can combine datasets matching features by a common key to add more data to its properties object. In our dataset, the key is each country's three-letter code. This can be done using a GIS application or a script.

Although not as efficient, this can also be done on the client side. In this step, we will load a CSV file (`data/un_regions.csv`) to obtain the HDI and area of each country and add these properties to each feature in the GeoJSON file.

The following code loads both files and processes each CSV row to keep just the properties we need: area, HDI, and the ISO 3166 three-letter code, used as the key:

```
const [data, themes] = await Promise.all([
  d3.json("../data/world/world-lowres.geojson"),
  d3.csv("../data/world/un_regions.csv", function(row) {
    return {
      area: +row.Area_km2,
      hdi: +row.HDI_2017,
      code: row.Code
    }
  })
]);
```

Next, we match the feature's id with the theme entry's code property to add three new properties to each feature's properties object: hdi, area and density. Some countries have -99 as their HDI in the source file, which means that they have no data. Here we replaced it with *undefined*:

The next step consists of matching the feature's id with the code property from each theme entry. This allows us to add three properties: hdi, area, and density to the each feature's properties object. An HDI of -99, indicates missing data, which we replaced with *undefined*.

```
data.features.forEach(function(d) {
  const entry = themes.filter(t => t.code === d.id)[0];
  if(entry) {
    d.properties.hdi = entry.hdi > 0 ? entry.hdi : undefined;
    d.properties.area = entry.area;
    d.properties.density = d.properties.pop / entry.area;
  }
});
```

Now you can display a thematic map comparing population density or HDI, simply replacing all occurrences of `d.properties.pop` with `d.properties.density` or `d.properties.hdi`.

What about letting the user choose the theme to show? This is easy. You just need to apply the new fill colors to the feature selection. The user can change this clicking a button. Create an array with the available themes and a key to select it:

```
const themesList = [
  {label: "Population", key: "pop"},
  {label: "Density", key: "density"},
  {label: "HDI", key: "hdi"}
];
```

Then, add a simple control panel with three buttons to apply the selected theme when clicked:

```
d3.select("body").append("div")
  .selectAll("button")
  .data(themesList)
  .join("button")
  .text(d => d.label)
  .on("click", (evt, d) => applyTheme(d.key));
```

The `applyTheme()` function computes the new extent based on the selected theme, chooses the best color scale for the data, then applies the color according to the new theme:

```
function applyTheme(t) {
  let colorScale;
  const extent = d3.extent(data.features, d => d.properties[t]);
  if (t === "hdi") { // use a different scale for HDI
    colorScale = d3.scaleSequential(d3.interpolateSpectral).domain(extent);
  } else {
    colorScale = d3.scaleSequentialPow(d3.interpolateRdYlBu).exponent(.2)
      .domain(extent.reverse());
  }
  svg.selectAll("path.country")
    .style("fill", d => colorScale(d.properties[t]));
}
```

Since HDI is a linear dataset, a sequential scale is best to represent it. Density and population work better with an exponential scale.

Step 7: Adding tooltips

The tooltip is a hidden `<g>` container containing a rectangle and two text elements that will be populated, moved and made visible when the user hovers the mouse over an object.

First, create some CSS styles for the rectangle and the text (in `tooltips.css`, imported by `main.css`):

```
#tooltip {
  pointer-events: none;
}
#tooltip rect {
  fill: white;
  fill-opacity: .7;
  stroke: black;
  stroke-width: .5;
}
#tooltip text {
  font-family: "Arial Narrow", sans-serif;
  font-size: 12px;
  text-anchor: middle;
}
```

To inform the category linked to each HDI value, add the following array:

```
const labels = ["No data", "Very Low", "Low", "Medium", "High", "Very High"];
```

The tooltip is a rounded rectangle (initially hidden) with two lines of text:

```
svg.append("g").attr("id", "tooltip").style("opacity", 0) // hidden
  .each(function(d) {
    d3.select(this).append("rect")
      .attr("height", 40)
      .attr("width", 150)
      .attr("rx", 5)
      .attr("ry", 5)
      .attr("x", -75)
      .attr("y", -20)
    d3.select(this).append("text").attr("y", -5)
    d3.select(this).append("text").attr("y", 15);
  });
```

In this example, we implemented all tooltip code in a separate module (`tooltips.js`), which is imported by the main script using the `tooltip` handle. Mouse hover handlers are registered for each country object, using `selection.on()`:

```
svg.selectAll("path.country")  
  .on("mouseenter", tooltip.show)  
  .on("mouseleave", tooltip.hide)
```

The event handler methods are shown below. The `show()` function moves the tooltip to the place where the mouse is positioned and makes it visible, updating the text elements with data obtained from the selected object:

```
export function show(evt, d) {  
  const tooltip = d3.select("#tooltip")  
    .attr("transform", `translate(${d3.pointer(evt)})`)  
    .style("opacity", 1);  
  tooltip.select("text:first-of-type").text(d.properties.name)  
  tooltip.select("text:last-child").text(valueString(d));  
}  
  
export function hide() {  
  d3.select("#tooltip").style("opacity", 0)  
}
```

Now, when your mouse hovers over a country, its HDI information will be displayed.

Step 8: Placing points on a map

The projection function used by a map's `geoPath` can be used to draw points on the map. This example loads the `data/cities1000.csv` file (`citiesFile`), which contains over 100 thousand locations on the earth. We will load and parse it along with the `shapesFile` containing the countries and select three properties:

```
const [data, cities] = await Promise.all([
  d3.json(shapesFile),
  d3.csv(citiesFile, function(row) {
    return {
      name: row.asciiname,
      position: [+row.longitude, +row.latitude],
      pop: +row.population
    }
  })
]);
```

A scale will be used to create circles that are proportional to the population:

```
const extent = d3.extent(cities, d => d.pop);
const areaScale = d3.scaleSqrt().domain(extent).range([0,5])
```

It will be used to determine the radius of each circle. The position of each circle must be computed using the same projection used for the map:

```
svg.selectAll("circle.city")
  .data(cities)
  .join("circle").attr("class", "city")
  .attr("cx", d => projection(d.position)[0])
  .attr("cy", d => projection(d.position)[1])
  .attr("r", d => areaScale(d.pop));
```

You can run the full code and view the result in `Step08/index.html`, which uses CSS to configure strokes, fills and fonts.

Another way to plot points is using GeoJSON *Point* objects. See the GeoJSON tutorial or the *Creating a dot density map* section in *Chapter 19*.

Step 9: Adding context with a graticule

A *graticule* is a set of parallel lines representing longitudes and latitudes. The `d3.geoGraticule()` function creates a generator that returns a 'MultiLineString' geometry object for these lines. A function that generates a graticule mesh with 10° spacing between lines can be created with:

```
const graticule = d3.geoGraticule();
```

Calling `graticule()` returns a GeoJSON object. Use it as the datum for a `<path>` element appended to your container, transformed by the `geoPath` function:

```
svg.append("path").attr("class","graticule").datum(graticule).attr("d", geoPath);
```

You can set styles using CSS:

```
.graticule { fill: none; stroke: blue; stroke-width: .5px }
```

To draw a thicker line along the equator, you can overlay another graticule, and configure its `step()` method to show a single latitude line:

```
const equator = d3.geoGraticule().step([0,90]); // latitude step every 90°
```

Apply a thicker stroke in CSS to make it stand out:

```
.equator { stroke: orange; stroke-width: 2px; }
```

Then render it:

```
svg.append("path").attr("class","equator").datum(equator).attr("d", geoPath);
```

Since both graticules are rendered with the same `geoPath` used for the countries, you can change the projection and they will adjust to it.

Step 10: Basic semantic zoom

You can quickly add simple zooming and panning to any map or chart with using the `d3.zoom()` behavior. First, create an SVG group as a container for the zoomable map:

```
const globe = svg.append("g").attr("class", "globe");
```

Then append the countries, the cities, and the graticules to this object (they were previously appended to `svg`):

```
globe.selectAll("path.country") // ...
globe.selectAll("circle.city") // ...
globe.append("path").attr("class", "graticule") // ...
globe.append("path").attr("class", "equator") // ...
```

Finally, set up the zoom behavior and call it from the globe's parent container (`svg`):

```
const zoom = d3.zoom()
    .translateExtent([[0,0],[dim.width, dim.height]])
    .scaleExtent([1, 30])
    .on('zoom', evt => semanticZoom(evt.transform));
svg.call(zoom);
```

The `semanticZoom()` function keeps stroke widths constant during zooming operations:

```
function semanticZoom(transform) {
    globe.attr("transform", transform);
    globe.selectAll("path.graticule, path.equator, path.country")
        .style("stroke-width", 0.5 / transform.k);
}
```

After adding this code you should be able to zoom in, zoom out and pan the map. Run `StepByStep/Step10/index.html` to see the result.

This is a very basic implementation using D3's zooming tools. The `d3-geo` module, however, offers geographic tools that can be used to translate, scale and rotate, which are much more efficient. See the *Zooming and panning* section in *Chapter 19*.

Step 11: Limiting the view to fit the map

Many maps only show a portion of the world. Your data may also be limited to that portion (e.g. the Madagascar map used in *the Drawing maps in D3* section) and rendering it using the default projection settings (intended for world maps) won't use the available space efficiently. We fixed that problem by centering and scaling the projection, but this approach is projection-dependent and not very intuitive. *Fitting* a feature in in a viewport is much simpler.

There are four different fitting methods that you can apply to a projection. The simplest one is `fitSize()`, which takes a `[width, height]` array, containing the dimensions of the viewport, and a GeoJSON object, which is the object you want to fit in the viewport.

Fitting overrides scaling and translating, but you can call these methods after you fit to make adjustments.

To fit the African continent in the viewport, we choose a center where the scaling will be applied, use the fitting method with the viewport's dimensions, and scale the map a bit (since our map also includes parts of Europe and Asia):

```
const projection = d3.geoMercator()
    .center([-40, -50])
    .fitSize([dim.width, dim.height], data)
    .scale(350);
```

You can also use `fitExtent()`, which takes an extent array. This will override the `center()` method.

Step 12: Using TopoJSON

Unlike GeoJSON, which duplicates shared polygon boundaries, TopoJSON breaks geometries into shared line segments called *arcs*. This reduces redundancy, produces smaller files, and simplifies operations such as creating meshes, merging geometries, and identifying neighbors.

The following code snippet reveals the general structure of the `data/world/world-lowres.topojson` file, generated from the GeoJSON file we have been using so far. It contains the same data but has only 102 kB, while the original file has 924 kB. The conversion was done via mapshaper.org.

```
{ "type": "Topology",
  "arcs": [[[25228,8916],[0,-58],[-45,-30],...], /* many more arc arrays... */ ],
  "objects": {
    "world": {
      "type": "GeometryCollection",
      "geometries": [
        { "type": "MultiPolygon", "id": "FJI"
          "properties": { "continent": "Oceania", "name": "Fiji", "pop": 889953 },
          "arcs": [[[0]], [[1]], [[2]] ],
        },
        /* ... many more geometry objects ... */
      ]
    }
  }
}
```

TopoJSON is JSON, so you can load it using `d3.json()`, but to use it in `d3.geoPath()` you need to convert its data to GeoJSON. This is possible with the `topojson-client` library, which is available via many CDN repositories. Add the following import to your file:

```
import * as topojson from "https://cdn.skypack.dev/topojson-client@3";
```

To draw the map, first load the TopoJSON dataset:

```
const tjdata = await d3.json('../data/world/world-lowres.topojson');
```

We will now use `topojson.feature()` to extract the data as a GeoJSON feature collection. This requires an application-dependent key created when the file was generated. Look for it in the `objects` property. For the `world-lowres.topojson` file (see the code snippet above), the key is `'world'`. The following will convert the data to GeoJSON:

```
const data = topojson.feature(tjdata, tjdata.objects['world']);
```

From here on, the code is the same as before, since `data` is a GeoJSON object.